

A Preregistered Audit of Dataset Contamination Controls in LLM-for-NetOps Benchmarks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired, confirmatory audit to test whether conservative deterministic contamination detectors (exact-match + fuzzy 5-gram + provenance heuristics) flag items in a reproducible 150-item NetOps sample and whether applying preregistered contamination controls changes validator-based success rates. Crucially, the executed sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") and shared generation semantics (independent_condition_generation = false), recorded in the archived model and task manifests; these two execution facts materially limited the audit’s sensitivity to generation-level contamination. No preregistered high-confidence contamination flags occurred and therefore no guarded exclusions or replacements were performed. All 150 baseline and matched guarded episodes passed the deterministic validator (baseline = 150/150, guarded = 150/150). The paired success-rate difference = 0.0 percentage points (95% CI [0.0, 0.0]); exact McNemar two-sided $p = 1.0$. We reconcile the preregistration→execution gap with artifact-grounded diagnosis, explain why the run is degenerate for detecting public-benchmark leakage, and provide concrete, artifact-anchored recommendations for audit designs that would be sensitive to contamination in web-sourced benchmarks.

I. INTRODUCTION

Motivation. Large language models (LLMs) are increasingly applied to NetOps tasks such as intent-to-configuration translation and automated configuration repair. Operational decisions based on benchmark results require that measured performance reflect model capability rather than dataset contamination or templated item leakage into model training. Meta-analyses and recent surveys call for contamination-aware evaluations and validator-backed oracles to avoid inflated reliability claims.

Research question and contribution. We preregistered and executed an artifact-anchored audit to answer: under conservative deterministic contamination detectors (exact normalized token equality; fuzzy 5-gram shingle overlap with preregistered thresholds; provenance timestamp heuristics) and a guarded exclusion policy, how many high-confidence flags arise in a reproducible 150-item NetOps sample, and how much does applying those controls change a single hosted LLM’s validator pass rate? Our contributions are: (1) a fully preregistered confirmatory experiment (study_id = contamination-audit-netops-2026-v1) executed end-to-end and archived; (2) exact, artifact-verified confirmatory statistics on $N_{\text{pairs}} = 150$ with deterministic validator traces; (3) an explicit preregistration→execution reconciliation that identifies why the run produced a degenerate null and prescribes artifact-grounded design changes to increase audit sensitivity.

Scope and bounded claims. The audit intentionally targeted a methodological question about preregistered contamination

controls, not an empirical prevalence estimate for public web-sourced benchmarks. The executed confirmatory sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") produced by the deterministic task generator and employed shared_initial_candidate generation semantics (independent_condition_generation = false). Because both facts are recorded in the archived model and task manifests, they limit inference: the observed zero-flag, zero-difference outcome applies to this synthetic sampling and generation regime and does not imply absence of contamination in independent public benchmarks.

II. RELATED WORK

LLM-driven NetOps systems and formal validators. Recent system and benchmark work documents the feasibility of translating intent into device configuration artifacts and of using LLMs as repair generators in controlled settings. Intent-driven configuration papers demonstrate that LLMs can produce syntactically plausible and often functionally correct configurations when tasks are constrained and prompts are carefully designed [7]. Benchmark and systems studies focused on configuration repair report that generator output quality improves when paired with deterministic validators or iterative validate–repair loops; these generate–then–verify patterns are an increasingly accepted design for raising practical reliability [1,3]. Complementary to these LLM-centric evaluations, classical network verification research (e.g., header-space analyses and localized subspecification approaches) supplies scalable, property-focused oracles for reachability and policy checks; these formal tools are practical building blocks for generate–validate pipelines that check functional correctness beyond surface syntactic match [4].

Agentic and integrated repair architectures. More recent work evaluates agentic or tool-augmented pipelines that combine LLM generation, retrieval/context, and external verifiers. Empirical comparisons show that agentic integrations with explicit verifier calls and iterative repairs typically reduce regressions relative to naive single-shot generation, but they do not eliminate per-case failures and their effectiveness depends on the verifier’s coverage and the agent’s tool contract [2]. System papers that emphasize tighter repair chains (localize→propose→validate→refine) report higher average repair rates on curated incident datasets, yet also highlight sensitivity to task heterogeneity, topology realism, and the exact verifier semantics used in evaluation [3]. These findings motivate using deterministic validators as the primary binary

oracle in contamination-aware audits, since validators operationalize functional pass/fail outcomes that matter to network operators.

Evaluation-validity critiques and contamination detection methods. Independent surveys and conceptual frameworks have raised two recurring risks for LLM evaluations relevant to NetOps: dataset contamination (items or paraphrases appearing in model training data) and construct invalidity (benchmarks dominated by low-complexity, templated prompts that overstate operational readiness) [8,6,5]. The literature recommends layered defenses: deterministic exact-match fingerprinting, fuzzy n-gram (shingle) overlap detectors with sensitivity analyses over multiple thresholds, provenance timestamp checks versus model training cutoffs, and human adjudication for borderline cases. These recommended practices inform our preregistered detector suite (exact canonical equality + fuzzy 5-gram overlap thresholds + provenance heuristics) and our guarded exclusion policy. Prior work also stresses preserving detector outputs (full overlap score distributions) to enable post-hoc assessment of threshold choices; this practical point motivated our archival design, and it underpins our recommendation to persist per-item overlap statistics even when no flags cross the preregistered thresholds [8].

Positioning relative to prior studies. Unlike broad capability benchmarks that report absolute performance across many models, our study is a preregistered, artifact-anchored audit that asks whether conservative, preregistered contamination controls would change validator-measured success on a reproducible sample under explicitly recorded execution semantics. Prior benchmark and system work assessed LLM repair efficacy and proposed integrated agentic architectures; contamination-awareness and detector design have been discussed in surveys and conceptual proposals but have seen fewer fully preregistered, verifier-backed confirmatory audits. Our contribution complements those lines by (a) operationalizing a conservative detector suite in a preregistration; (b) using a deterministic verifier as the functional oracle; and (c) providing an artifact-grounded reconciliation when preregistered choices (synthetic provenance + shared generation semantics) rendered the confirmatory paired test degenerate. This positioning clarifies that our findings diagnose audit design sensitivity rather than claiming empirical prevalence of contamination in independent web-sourced benchmarks.

III. METHODOLOGY

Preregistration and primary estimand. The experiment was preregistered (study_id = contamination-audit-netops-2026-v1) and froze analysis and exclusion rules before execution. The primary estimand is the paired success-rate difference (guarded - baseline) across item×model pairs, tested by exact McNemar two-sided test and summarized with a paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) as specified in the preregistered paired_binary_analysis_v1 executor.

Detectors and guarded policy (preregistered). The preregistered detectors were: (1) exact normalized token equality

after canonicalization; (2) fuzzy 5-gram shingle overlap (Jaccard/overlap) with preregistered high-confidence threshold ≥ 0.80 and sensitivity thresholds at 0.65 and 0.50; and (3) provenance timestamp heuristics comparing item timestamps to model cutoff. The guarded policy deterministically excludes only high-confidence flagged items (exact match OR fuzzy overlap ≥ 0.80) and replaces them via deterministic retemplating/topology augmentation from the same generator with fixed seeds; replacements must meet an embedding similarity constraint per the preregistration.

Sampling, generation semantics, and the critical execution choice. The preregistration allowed deterministic task generators for reproducibility and permitted shared generation semantics when appropriate. The executed run used deterministic_netops_task_generator_v5 to produce synthetic tasks (source_identifier prefix "synthetic-netops:") for the confirmatory sample. Execution settings used shared_initial_candidate semantics (independent_condition_generation = false): when no exclusions occur, one shared model generation per item is reused for both baseline and guarded episodes. These execution settings are recorded in the archived model_configuration and task_manifest artifacts. The preregistered contract also planned replacement rules, sensitivity analyses, and a maximum of 300 model calls (one per condition per item) if independent generations were used.

Verifier and scoring rule. The verification oracle is deterministic_netops_validator_v5. For intent→configuration tasks the validator applies parse→state-update→property checks and returns a binary pass/fail along with detailed traces (observed_touched_field_count, parsed_command_count, required_command_count, violation codes). The preregistered scoring rule is full_intent_constraint_validation: outputs must achieve the intended final state and respect workflow constraints. Validator traces are archived for every episode and provide the empirical basis for the binary outcomes used in the paired analysis.

Planned sensitivity and power. The preregistration included sensitivity analyses: treating failed model calls as failures; evaluating fuzzy thresholds at 0.50, 0.65, 0.80; and subgrouping by templated vs realistic items. The power guidance documented that a multi-model or two-call per item design (300 independent generations) would detect smaller paired differences (≈ 4 –6 percentage points under specified assumptions), while the single-model single-call shared-generation regime has reduced sensitivity. These planning notes are part of the preregistration archive and informed our interpretation of the executed null result.

IV. RESULTS

Execution accounting and deterministic reconciliation. Execution completed planned_episode_count = 300 episodes (150 item pairs) with completed_episode_count = 300 and failed_episode_count = 0. Because independent_condition_generation = false and no high-confidence exclusions occurred, the pipeline used model_calls_used = 150 (one shared generation per item)

rather than two separate generations per condition; provider call accounting and pair reconciliation artifacts confirm these counts and consistency (`complete_pair_count` = 150; `n_11` = 150; `n_10` = `n_01` = `n_00` = 0).

Contamination detectors: absence of preregistered high-confidence flags. The preregistered detectors produced zero high-confidence contamination flags for the executed sample. The `contamination_summary` artifact is empty of flagged rows; the detector workflow archived flagging events and replacement traces when and where they occurred, and no such events were created in this run. Importantly, the detector implementation recorded detailed per-item fuzzy overlap outputs only when thresholded flags or replacement handling were invoked; because no items met the high-confidence threshold, an aggregated distribution of all per-item overlap scores is not present in the archive. We therefore report the artifact-documented absence of high-confidence flags and the absence of per-item overlap summaries rather than invent per-item overlap statistics.

Validator outcomes and confirmatory statistics. The `deterministic_netops_validator_v5` returned pass for all baseline episodes (`baseline_success_count` = 150/150; `baseline_success_rate` = 1.0) and for all guarded episodes (`guarded_success_count` = 150/150; `guarded_success_rate` = 1.0). The paired contingency table is `n11` = 150, `n10` = 0, `n01` = 0, `n00` = 0. The primary estimand (`paired_success_rate_difference_guarded_minus_baseline`) = 0.0 percentage points. Paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) = [0.0, 0.0]. Exact McNemar two-sided p = 1.0. All per-episode validator traces and raw model responses are archived and form the empirical basis for these numbers.

Representative episode diagnostic. A representative archived episode (`pair-000001` / `task-000001`) illustrates the structure and traces: reference and normalized response are identical ("`interface edge1 admin down\ninterface edge1 vlan 30\ninterface edge1 mtu 1600\ninterface edge1 admin up`"); `validator_parsed_command_count` = 4, `required_command_count` = 4, and `score` = 1 (`VALID_CONFIGURATION`). The archived `validator_trace` records initial and final state snapshots and the sequence of applied configuration commands; this exemplar is included in the execution bundle as a compact verification of scorer behavior.

Missing overlap summaries and interpretation. The preregistered fuzzy-overlap detector would have produced per-item overlap scores and flags if overlaps met preregistered thresholds. In this execution the detector produced no high-confidence flags and therefore the contamination detector workflow did not persist a full per-item overlap score table. This is an explicit artifact-grounded absence: the detector logged no threshold crossings or replacements and therefore no aggregated overlap distribution is available in the archive for reporting here. We discuss the implications of this missing summary in the Threats and Recommendations sections.

V. DISCUSSION

Why the result is degenerate: artifact-grounded diagnosis. Two archived execution facts explain the degenerate null outcome. First, the sampled items were provenance-stable synthetic tasks generated by `deterministic_netops_task_generator_v5` (`source_identifier` prefix "`synthetic-netops:`"), so canonicalized exact matches to public corpus fingerprints and high fuzzy 5-gram overlaps above the preregistered high-confidence threshold were unlikely. Second, execution semantics set `independent_condition_generation` = `false` (`shared_initial_candidate`), so when no exclusions were triggered a single shared generation per item was reused for baseline and guarded episodes. Both facts are recorded in the archived `model_configuration` and `task_manifest` artifacts and together produced identical baseline and guarded outputs per item, making the paired comparison uninformative for generation-level contamination detection in this run.

What this execution answers and what it does not. The archived evidence answers the preregistered methodological question for the executed sampling regime: given provenance-stable synthetic sampling and conservative deterministic detectors, do preregistered high-confidence flags arise and do deterministic guarded exclusions alter validator pass rates? For this sample and settings the answer is no. The result does not generalize to web-sourced public benchmarks, paraphrase-sensitive contamination patterns, independent re-generation semantics, other LLM families, or multi-model designs. We therefore avoid extrapolating beyond the archive-supported conclusions.

Audit sensitivity, power, and practical tradeoffs. The pre-registration documented that a two-generation per item design (`independent_condition_generation` = `true`) or a multi-model plan increases the number of independent trials and therefore the power to detect paired differences of modest size (target ≈ 5 percentage points under conservative assumptions). The executed single-model, shared-generation regime reduces independent variance and makes detection of generation-level contamination implausible unless contamination flags arise deterministically. Practically, contamination audits that aim to detect web-sourced leakage should prioritize provenance-rich sampling, independent generations per condition, and multi-model replication to increase observable discordance and power.

Concrete artifact-grounded recommendations. Based on pre-registration, execution artifacts, and the deterministic reconciliation, we recommend: (1) sample provenance-rich public items (URLs/DOIs and timestamps) to exercise detector sensitivity to real-world leakage; (2) set `independent_condition_generation` = `true` so baseline and guarded conditions are produced independently and cannot collapse to shared outputs; (3) expand to multiple model instances and independent seeds to reveal cross-model contamination signals; (4) persist per-item fuzzy overlap scores regardless of flag status to enable overlap distribution reporting; and

(5) add paraphrase-robust detectors and human adjudication for borderline cases. Each recommendation follows from artifacts: the execution manifest and deterministic_reconciliation confirm the current run’s settings and limits, and the empty contamination_summary motivates broader sampling and independent generation in future audits.

Operational implications for NetOps. For practitioners, the audit demonstrates that audit sensitivity depends critically on sampling provenance and generation semantics. Verifier-backed validators are valuable oracles for functional correctness checks, but auditor designs must ensure samples exercise detector capabilities and record detector outputs at sufficient granularity for post-hoc analysis. In production-risk settings, independent generation, cross-model checks, and human adjudication remain advisable.

VI. LIMITATIONS

- Single-model execution (gpt-5-mini) — results do not generalize across other LLM families, sizes, or training corpora.
- Sampled items were provenance-stable synthetic/deterministically generated tasks (source_identifier prefix "synthetic-netops:") for this run; absence of high-confidence flags here may not reflect contamination prevalence in real-world, web-crawled benchmarks.
- Generation semantics used shared_initial_candidate across baseline and guarded conditions (independent_condition_generation = false), producing identical model outputs when no exclusions occurred and thus limiting sensitivity to interventions that rely on independent re-generation.
- Contamination detection relied on preregistered conservative heuristics (exact match and fuzzy 5-gram overlap thresholds); subtler contamination patterns (paraphrased memorization, partial leakage) may escape these heuristics and require richer provenance or human adjudication.
- Passing the deterministic_netops_validator_v5 indicates satisfaction of the checked functional constraints but does not imply comprehensive production readiness or absence of semantic regressions outside the validator’s scope.

VII. CONCLUSION

We executed a fully preregistered, artifact-anchored paired audit of conservative contamination detectors for LLM-for-NetOps evaluation. In the reproducible 150-pair synthetic sample we observed zero preregistered high-confidence contamination flags and identical validator pass rates (150/150 baseline, 150/150 guarded). The degenerate null outcome is artifact-supported and stems from provenance-stable synthetic sampling and shared generation semantics; it does not imply absence of contamination in web-sourced benchmarks or failure of the preregistered detectors in different sampling regimes. We provide an explicit preregistration→execution reconciliation, confirm deterministic reconciliation checks passed, and recommend concrete design changes—provenance-rich

sampling, independent generation per condition, multi-model scope, paraphrase-sensitive detectors, and matched power calculations—to increase audit sensitivity in future studies. All experiment artifacts, validator traces, analysis inputs/outputs, and execution manifests are archived and available as recorded in the Disclosure Statement below.

DISCLOSURE STATEMENT

This study was executed under the autonomous-run preregistration contamination-audit-netops-2026-v1 (preregistration SHA-256 recorded in the archived bundle). LLM used: gpt-5-mini (declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic task generator: deterministic_netops_task_generator_v5 (version 5.0). Deterministic validator: deterministic_netops_validator_v5 (version 5.0). Representative executed artifacts (by path) include execution/model_configuration.json, execution/task_manifest.jsonl, execution/raw_results.jsonl, analysis/paired_contingency_table.csv, and analysis/contamination_summary.csv; the full execution bundle contains raw responses, validator traces, execution logs, and the transformation manifest. Exact immutable initial master-prompt reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Topic constraints (Generative AI and LLMs for NetOps) were selected by the human Research Director prior to the autonomy lock. After the autonomy lock, the AI system autonomously performed literature search, gap selection, preregistration, experiment execution, analysis, manuscript drafting, peer-review simulation, and revision. All generation prompts, model outputs, validator traces, sampling seeds, replacement rules, execution logs, and analysis artifacts were produced and archived by the autonomous pipeline and are included in the execution bundle. No manual human editing of technical content, analyses, references, figures, tables, captions, or the Disclosure Statement occurred after the autonomy lock. Pre-lock development and rehearsal runs occurred (permitted) but pre-lock artifacts were not used as empirical evidence for this final study unless re-created under the locked pipeline. The execution followed preregistered missingness, retry, and exclusion policies; no deviations occurred.

REFERENCES

- [1] <https://arxiv.org/abs/2604.22513>
- [2] <https://doi.org/10.48550/arxiv.2606.06212>
- [3] <https://doi.org/10.48550/arxiv.2605.22092>
- [4] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>
- [5] <https://doi.org/10.1007/s11704-026-60308-3>
- [6] Buğra Alperen Ulurmak, Rifat Kurban, "EvalSafetyGap: A Hybrid Survey and Conceptual Framework for LLM Evaluation-Safety Failures", 2026
- [7] Nguyen Van Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, Ting Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", 2024, 10.1145/3703155